Definition of transformers

GLyC

December 12, 2025

During this work we wont use the definition of transformers from Li, et all. We will use similar one but equivalent. The structure will be the same but we will make more explicits the conditions for accepting and rejecting. Also the condition for stopping the CoT will be more clear.

As we said, the structure will be the same but we will change the alphabet. We will require that it has two special characters that represents the acceptance and rejection of the words. Also this characters should not appear in the input word. We will say that a transformer accepts a word when it prints the special character for acceptance and that it rejects a word when it prints the one for rejection.

probably we can eliminate this precondition somehow

Note that with this definition the CoT stops when it prints any of the two stopping characters. The transformers of Li, et all can easily simulate the ones defined by us because we can force the transformer through the token embedding so that when it sees a stopping character it replicates it until it reaches n_{max} and then it prints a 0 or a 1. In the same way we can force through the positional embedding to print a stopping character when the n_{max} step is reached.

this looks super doable but maybe it requires some thinking

Probabilistic

We will define the probabilistic transformers in the exact same way as the deterministic transformers with the exception that in the last step of each iteration of CoT we will replace the *argmax* function by a *sample* function witch will output a token not deterministically based on the probability distribution computed in the step before.

We will say that a probabilistic transformer PTF computes a language L when

$$x \in L \iff \mathbb{P}_{out_{T(|x|)} \dots out_1}(out_{T(|x|)} | out_{T(|x|)} \dots out_1) > 2/3$$

An alternative definition would be based on the random bits it needs to do the sampling.

We can simulate the sample function with the following algorithm. First we define the ranges of each token storing their upper end and then we call the function sample starting with s being $0.\_$:

1. sample(ranges, $0.b_1, \dots, b_k$)

2. s $= 0.b_1, \ldots, b_k, b_{k+1}$ // add random bit to s

3. if exists i s.t. $ranges[i] =_k s$ return sample(ranges, s)

4. else return $token[max\{j \mid ranges[j] \leq s\}]$

Lets see how to define the ranges. Lets assume the tokens $\Sigma = \{t_1, \cdots t_n\}$ and their associated probabilities $p_1, \cdots p_n$. The range associated to the ith token will be $[\Sigma_{j=1}^{i-1} p_j, p_i + \Sigma_{j=1}^{i-1} p_j)$ which is the same as $[\Sigma_{j=1}^{i-1} p_j, \Sigma_{j=1}^{i} p_j)$. It is important to notice that this calculations are made outside the transformer, there fore are not limited to the floating point arithmetic.

For example, with the tokens $\Sigma = \{t_1, t_2, t_3\}$ and the probabilities $\{p_1 = 0.33, p_2 = 0.22, p_3 = 0.45\}$ the ranges will be $[0, 0.33)$ for x_1, $[0.33, 0, 55)$ for x_2 and $[0.55, 1)$ for x_3.

is it worth to formally prove that this is sound?

An important question to make it's how many bits are required for each sampling. In the worst case are needed as many as bits has the smallest probability, because the those numbers came from the transformer, they are bounded by its finite representation. It's important to notice that although that number its bounded it could be exponential in relation to the size of the representation because we will need the exact binary representation of the number, not in floating point representation.

could happen that by the numerical error the probabilities doesnt add to 1?

Its important to notice that the expected number of bits, although in the worst case it could be exponential, its constant, only depends on the number of tokens. This is because the probability of reaching the k-th recursive call its the probability of matching any border of the ranges so its $\sum_{i=1}^{|\Sigma|} \frac{1}{2^k} = \frac{|\Sigma|}{2^k}$. Lets call X the random variable representing the number of recursive steps:

here it goes an argument about why the expected number of bits needed its not exponential

$$E[X] = \sum_k Pr(X = k)k \leq \sum_k Pr(X = k)k = \sum_k \frac{\mid \Sigma \mid}{2^k} k =\mid \Sigma \mid \sum_k \frac{k}{2^k} =\mid \Sigma \mid 2$$

Then because in each recursive step its needed only one random bit, the expected number of bits needed for each sample its equal to the doble of the number of tokens. Because those are not changing, we say that the expected number of random bits needed its constant.

In this definition we will say that given a random list of bits of size at most ?? the transformer will answer right with a high probability.

$$x \in L \iff \mathbb{P}_{r=\{0,1\}^*}(PTF(x, r)) > 2/3$$

If the number representation it is fixed, then the number of bits needed it is linear with relation to the number of CoT steps.